

Experiment # 06

Implementation of Matrix Operations using OpenMP

Objective:

The objective of this lab is to get the knowledge about OpenMP parallel computation techniques.

Software Tools:

✓ Ubuntu

Theory:

What is OpenMP?

OpenMP (Open Multi-Processing) is an application programming interface (API) that supports multi-platform shared memory multiprocessing programming in C, C++, and Fortran, on most platforms, instruction set architectures and operating systems, including Solaris, AIX, HP-UX, Linux, MAC-OS, and Windows. It consists of a set of compiler directives, library routines, and environment variables that influence runtime behavior.

OpenMP is managed by the nonprofit technology consortium OpenMP Architecture Review Board (or OpenMP ARB), jointly defined by a group of major computer hardware and software vendors, including AMD, IBM, Intel, Cray, HP, Fujitsu, NVidia, NEC, Red Hat, Texas Instruments, Oracle Corporation, and more.

OpenMP uses a portable, scalable model that gives programmers a simple and flexible interface for developing parallel applications for platforms ranging from the standard desktop computer to the supercomputer.

OpenMP uses the fork-join model. It is both directive- and library-based. OpenMP threads share a single executable, global memory, and heap (malloc, new). Each OpenMP thread has its own stack (function arguments, function return address, local variables).

Using OpenMP requires no dramatic code changes. It probably gives you the biggest multithread benefit per amount of work you have to put in to using it.

An application built with the hybrid model of parallel programming can run on a computer cluster using both OpenMP and Message Passing Interface (MPI), such that OpenMP is used for parallelism within a (multi-core) node while MPI is used for parallelism between nodes. There have also been efforts to run OpenMP on software distributed shared memory systems, to translate OpenMP into MPI and to extend OpenMP for non-shared memory systems.

OpenMP: Basic Library-Functions & Compiler-Directives

Two Library functions:

- `int omp_get_thread_num (); // get ThreadID`
- `int omp_get_num_threads (); // get ThreadsCount`

Parallel Directive in OpenMP

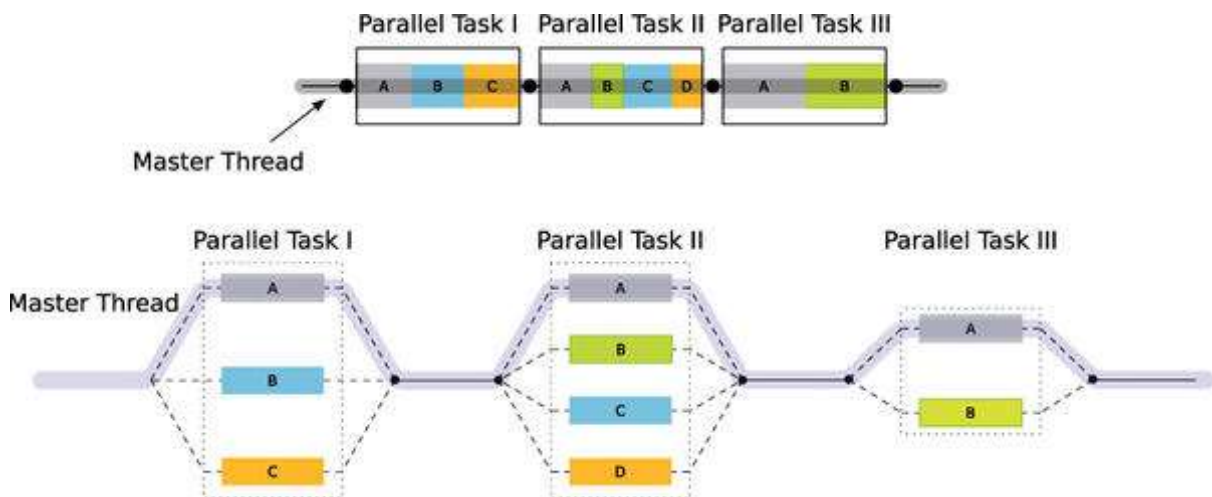
- Parallel directive in openMP creates group of threads which work concurrently on next structured block:

```
#pragma omp parallel [clause list]
```

```
/* structured block */
```

- OpenMP programs execute serially until they encounter the parallel directive, which creates a group of threads.
- The main thread that encounters the parallel directive becomes master of this group of threads & is assigned the thread id=0 within the group.
- Barrier for all the thread at the end of parallel block is implicitly implied

OpenMP programming Model



Parallel Directive Clause List

The clause list with parallel directive is used to specify degree of concurrency & data handling.

Degree of Concurrency:

The clause `num_threads(int expr)` specifies the number of threads to be created.

Data Handling:

- Clause `shared (variable list)` indicates the variables that are shared across all the threads.
- Clause `private (variable list)` indicates variables local to each thread.

- Clause firstprivate (variable list) is similar to the private, except values of variables are initialized to corresponding variable values before the parallel directive.
- Clause option for default(shared) or default(private) also allowed.

OpenMP Introduction & Installation

After a long thirst for parallelizing highly regular loops in matrix-oriented numerical programming, OpenMP was introduced by “OpenMP Architecture Review Board (ARB) on 1997”. In the subsequent releases, the enthusiastic OpenMP team added many features to it including the task parallelizing, support for accelerators, user-defined reductions and lot more. The latest “OpenMP 5.0” release was made in 2018 November.

Open Multi-processing (OpenMP) is a technique of parallelizing a section(s) of C/C++/Fortran code. OpenMP is also seen as an extension to C/C++/Fortran languages by adding the parallelizing features to them. In general, OpenMP uses a portable and scalable model that gives programmers a simple and flexible interface for developing parallel applications for platforms that ranges from the normal desktop computer to the high-end supercomputers.

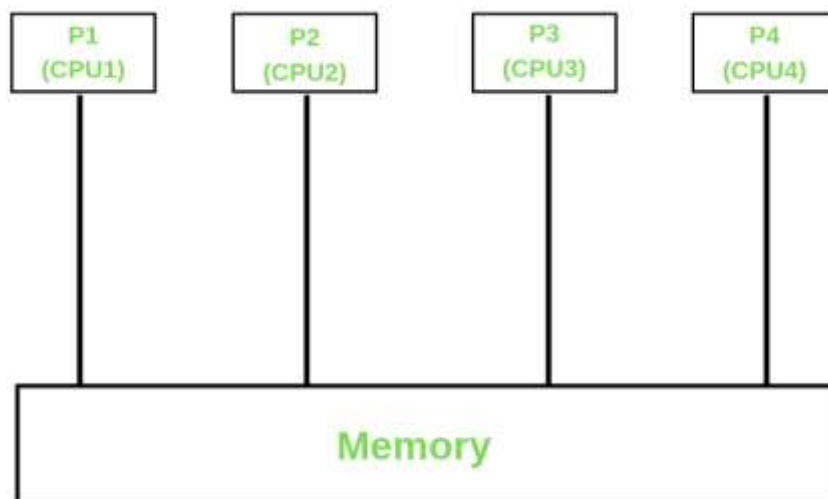
Thread VS Process

A process is created by the OS to execute a program with given resources(memory, registers); generally, different processes do not share their memory with another. A thread is a subset of a process, and it shares the resources of its parent process but has its own stack to keep track of function calls. Multiple threads of a process will have access to the same memory.

Parallel Memory Architectures

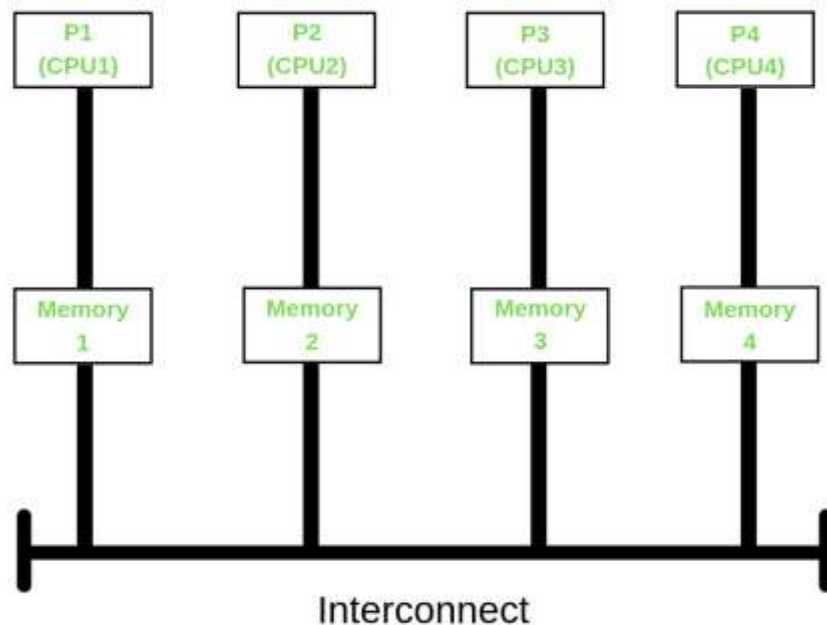
Before getting deep into OpenMP, let's revive the basic parallel memory architectures. These are divided into three categories;

- **Shared Memory**



OpenMP comes under the shared memory concept. In this, different CPU's (processors) will have access to the same memory location. Since all CPU's connect to the same memory, memory access should be handled carefully.

➤ Distributed Memory



Here, each CPU (processor) will have its own memory location to access and use. In order to make them communicate, all independent systems will be connected together using a network. MPI is based on distributed architecture.

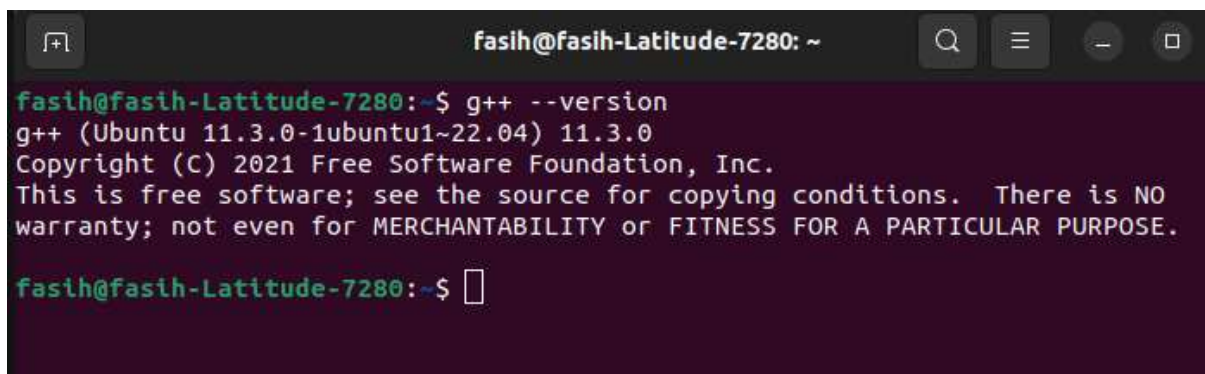
➤ Hybrid

Hybrid is a combination of both shared and distributed architectures. A simple scenario to showcase the power of OpenMP would be comparing the execution time of a normal C/C++ program and the OpenMP program.

Setting up OpenMP on Ubuntu

1. Run `sudo apt-get install libomp-dev` in your Terminal.
2. Create a C++ Project , and title it `HelloOpenMP` .
3. Select your project, and go to the `Properties` dialog.
4. Go to `C/C++ Build -> Settings` .
5. Select `GCC C++ Compiler / Miscellaneous` .
6. In the `Other flags` input, add on `-fopenmp` .
7. Select `GCC C++ Linker / Libraries` .
8. In the `Libraries (-l)` field, click the add button and type in `gomp`.

Step 1: Check the G++ version of the compiler

A terminal window with a dark background. The prompt is 'fasih@fasih-Latitude-7280: ~'. The user enters 'g++ --version'. The output is: 'g++ (Ubuntu 11.3.0-1ubuntu1~22.04) 11.3.0', 'Copyright (C) 2021 Free Software Foundation, Inc.', 'This is free software; see the source for copying conditions. There is NO warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.', and the prompt 'fasih@fasih-Latitude-7280:~\$' with a cursor.

```
fasih@fasih-Latitude-7280:~$ g++ --version
g++ (Ubuntu 11.3.0-1ubuntu1~22.04) 11.3.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

fasih@fasih-Latitude-7280:~$
```

G++ provides support for OpenMP starting from its version 4.2.0. So if the system has G++ compiler with the version higher than 4.2.0, then it must have OpenMP features configured with it.

If the system doesn't have the G++ compiler, we can use the following command:

```
Sudo apt install g++
```

STEP 2: Configuring OpenMP

We can check whether the OpenMP features are configured into our compiler or not, using the command

```
echo |cpp -fopenmp -dM |grep -i open
```

A terminal window with a dark background. The prompt is 'fasih@fasih-Latitude-7280: ~'. The user enters 'g++ --version'. The output is: 'g++ (Ubuntu 11.3.0-1ubuntu1~22.04) 11.3.0', 'Copyright (C) 2021 Free Software Foundation, Inc.', 'This is free software; see the source for copying conditions. There is NO warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.', and the prompt 'fasih@fasih-Latitude-7280:~\$' with a cursor. The user then enters 'echo |cpp -fopenmp -dM |grep -i open'. The output is: '#define _OPENMP 201511' and the prompt 'fasih@fasih-Latitude-7280:~\$' with a cursor.

```
fasih@fasih-Latitude-7280:~$ g++ --version
g++ (Ubuntu 11.3.0-1ubuntu1~22.04) 11.3.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

fasih@fasih-Latitude-7280:~$ echo |cpp -fopenmp -dM |grep -i open
#define _OPENMP 201511
fasih@fasih-Latitude-7280:~$
```

If OpenMP is not featured in the compiler, we can configure it use using the command:

```
sudo apt install libomp-dev
```

Running first program in OpenMP

Code

A code editor window with a dark background. The title bar shows 'test.cpp' and a close button. The code is: '1 #include<iostream>', '2 #include<omp.h>', '3', '4 using namespace std;', '5'.

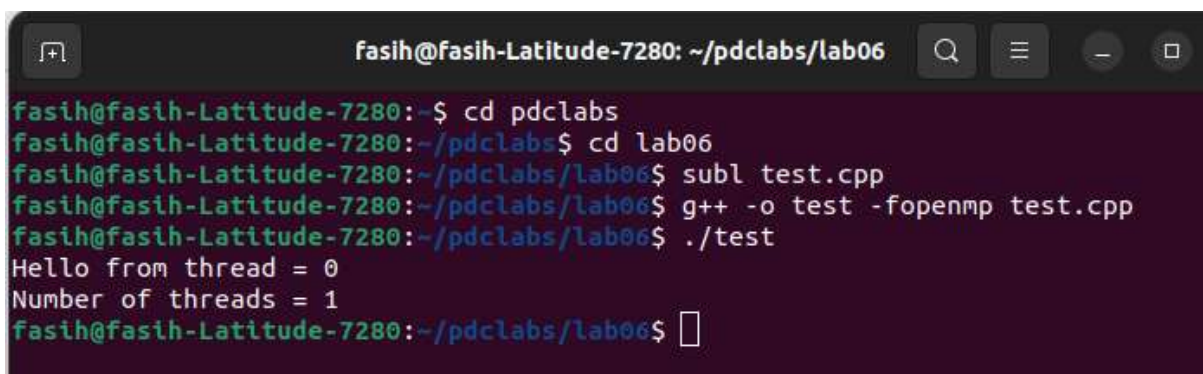
```
test.cpp
1  #include<iostream>
2  #include<omp.h>
3
4  using namespace std;
5
```

```

6  int main (void)
7  {
8      int nthreads;
9      int tid;
10
11     tid = omp_get_thread_num();
12
13     cout<<"Hello from thread = "<<tid<<"\n";
14
15     if(tid==0)
16         nthreads = omp_get_num_threads();
17
18     cout<<"Number of threads = "<<nthreads<<"\n";
19
20     return 0;
21 }

```

Output



```

fasih@fasih-Latitude-7280: ~/pdclabs/lab06
fasih@fasih-Latitude-7280:~$ cd pdclabs
fasih@fasih-Latitude-7280:~/pdclabs$ cd lab06
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ subl test.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ g++ -o test -fopenmp test.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ ./test
Hello from thread = 0
Number of threads = 1
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ 

```

Compile

`g++ -o [.exe file] -fopenmp [.cpp file]`

Execute

`./[.exe file]`

Steps to create a Parallel Program

Include the header file: We have to include the OpenMP header for our program along with the standard header files.

```

//OpenMP header
#include <omp.h>

```

Specify the parallel region: In OpenMP, we need to mention the region which we are going to make it as parallel using the keyword **pragma omp parallel**. The `pragma omp parallel` is used to fork additional threads to carry out the work enclosed in the parallel. The original thread will be denoted as the master thread with thread ID 0. Code for creating a parallel region would be: -

```
#pragma omp parallel
{
    //Parallel region code
}
```

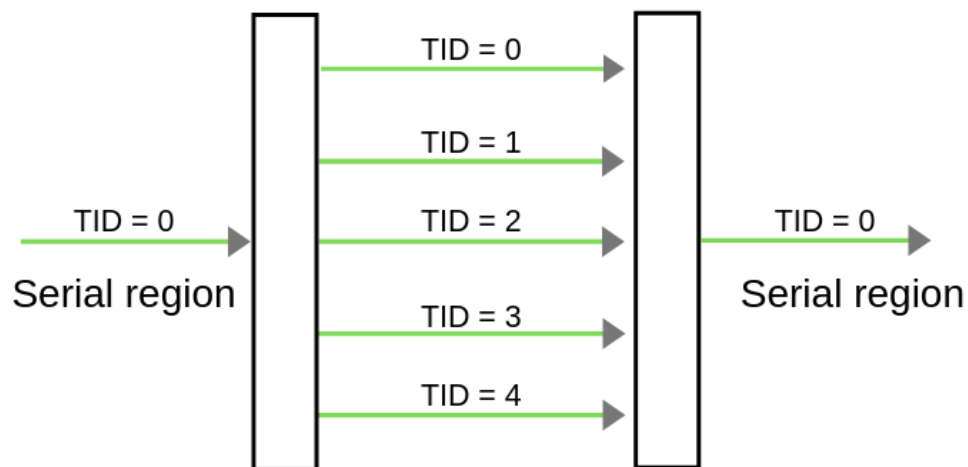
We include: -

```
#pragma omp parallel
{
    printf("Hello World... from thread = %d\n", omp_get_thread_num());
}
```

Set the number of threads:

We can set the number of threads to execute the program using the external variable.

```
export OMP_NUM_THREADS=5
```



Code

```
test1.cpp
1  #include<omp.h> // OpenMP header
2  #include<iostream>
3
4  using namespace std;
5
6  int main (void)
7  {
8
9      #pragma omp parallel
10     {
11         cout<<"\nHello from thread = "<<omp_get_thread_num();
12     }
13
14     return 0;
15 }
```

Output


```
fasih@fasih-Latitude-7280: ~/pdclabs/lab06
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ subl test1.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ export OMP_NUM_THREADS=5
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ g++ -o test1 -fopenmp test1.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ ./test1

Hello from thread = 4
Hello from thread = 1
Hello from thread = 2
Hello from thread = 3
Hello from thread = 0fasih@fasih-Latitude-7280:~/pdclabs/lab06$
```

OR

Code

```
test4.cpp x test1.cpp x
1  #include<iostream>
2  #include<omp.h>
3
4  using namespace std;
5
6  int main ()
7  {
8      #pragma omp parallel num_threads(5)
9      {
10         cout<<"\nHello from thread# "<<omp_get_thread_num();
11     }
12     return 0;
13 }
```

Output

```
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ subl test4.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ g++ -o test4 -fopenmp test4.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ ./test4

Hello from thread# 4
Hello from thread# 1
Hello from thread# 2
Hello from thread# 3
Hello from thread# 0fasih@fasih-Latitude-7280:~/pdclabs/lab06$
```

We can set the number of threads that we want to execute, using the following command:

```
export OMP_NUM_THREADS=8
```

To accept this number of threads, we can use the command:

```
int no_of_thread = omp_get_num_threads();
```

We can either define a separate variable to store the number of threads or we can directly use:

omp_get_num_threads()

Note: The `omp_get_num_threads()` returns the number of threads in the current task. However, the `omp_get_thread_num()` returns the thread number of the calling thread. There's a difference between the two.

In C/C++, OpenMP uses `#pragmas` to indicate the beginning of the parallel region. The OpenMP “parallel for” flag is used to split the task of the program among the threads that the OpenMP system is working on.

OpenMP Barrier

Code: -

```
test2.cpp
1  #include<iostream>
2  #include<omp.h>
3  using namespace std;
4
5  int main ()
6  {
7      int id;
8      int nthreads;
9
10     #pragma omp parallel private(id)
11     {
12         id=omp_get_thread_num();
13         cout<<"\nHello World from thread# "<<id;
14
15         if(id==0)
16         {
17             nthreads=omp_get_num_threads();
18             cout<<"\nThere are total "<<nthreads<<" threads"<<endl;
19         }
20     }
21     return 0;
22 }
23 }
```

Output: -

```
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ subl test2.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ g++ -o test2 -fopenmp test2.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ ./test2

Hello World from thread# 0
There are total 4 threads

Hello World from thread# 1
Hello World from thread# 2
Hello World from thread# 3fasih@fasih-Latitude-7280:~/pdclabs/lab06$
```

Code: -

```
test2.cpp
1  #include<iostream>
2  #include<omp.h>
3  using namespace std;
4
5  int main ()
6  {
7      int id;
8      int nthreads;
9
10     #pragma omp parallel private(id)
11     {
12         id=omp_get_thread_num();
13         cout<<"\nHello World from thread# "<<id;
14
15         #pragma omp barrier
16         {
17             if(id==0)
18             {
19                 nthreads=omp_get_num_threads();
20                 cout<<"\nThere are total "<<nthreads<<" threads"<<endl;
21             }
22         }
23     }
24     return 0;
25 }
26 }
```

Output: -

```
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ subl test2.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ g++ -o test2 -fopenmp test2.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ ./test2

Hello World from thread# 0
There are total 4 threads

Hello World from thread# 1
Hello World from thread# 2
Hello World from thread# 3fasih@fasih-Latitude-7280:~/pdclabs/lab06$ subl test2.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ g++ -o test2 -fopenmp test2.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ ./test2

Hello World from thread# 0
Hello World from thread# 2
Hello World from thread# 1
Hello World from thread# 3
There are total 4 threads
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ □
```

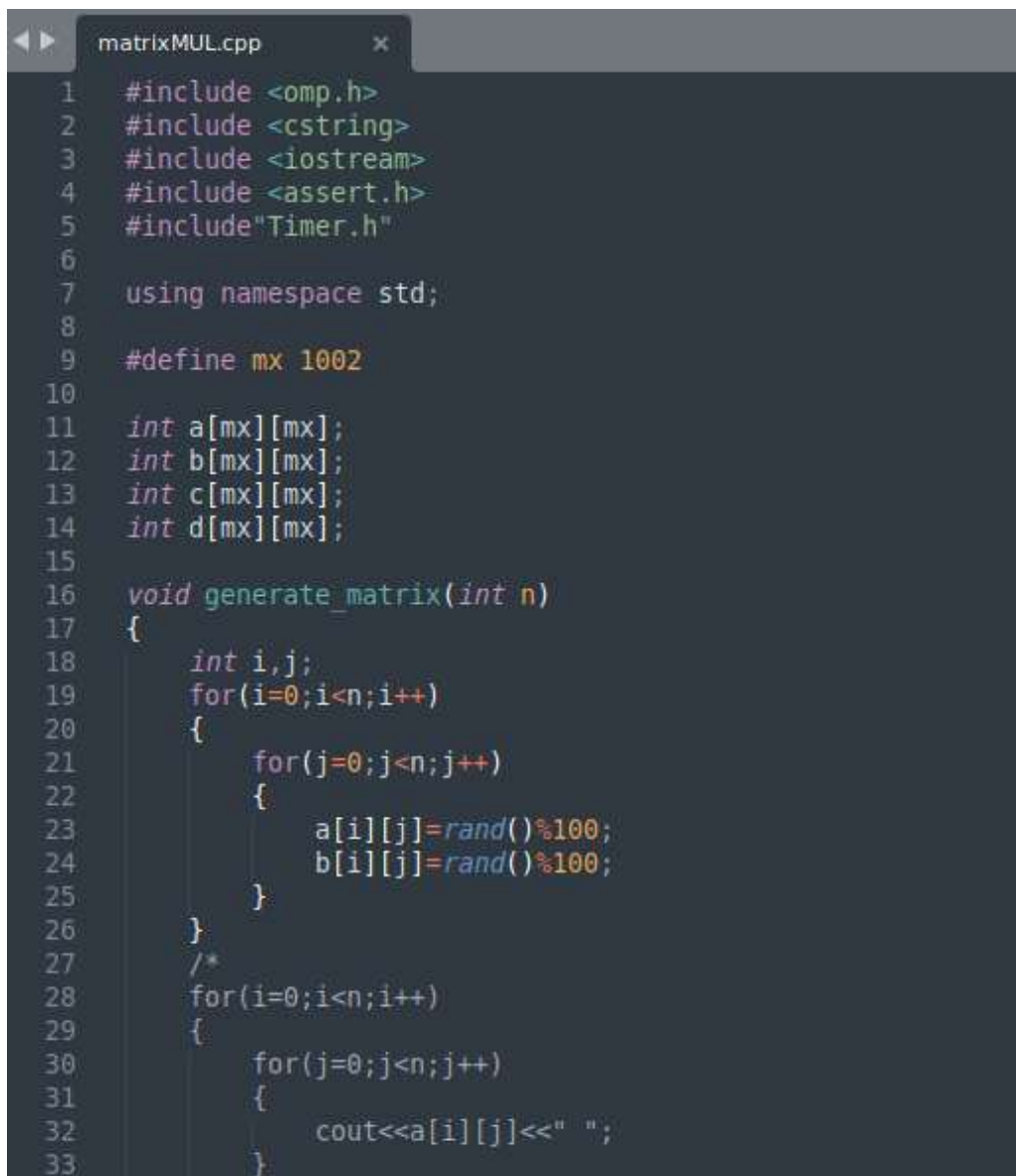
Example: -

```
#pragma omp parallel if (is_parallel == 1) num_threads(8) shared
(var_b)
private (var_a) firstprivate (var_c) default (none)
{
```

```
/* structured block */  
}
```

- `if (is_parallel == 1) num_threads(8)`
 - If the value of the variable `is_parallel` is one, create 8 threads
- `shared (var_b)`
 - Each thread shares a single copy of variable `b`
- `private (var_a) firstprivate (var_c)`
 - Each thread gets private copies of variable `var_a` and `var_c`
 - Each private copy of `var_c` is initialized with the value of `var_c` in main thread when the parallel directive is encountered
- `default (none)`
 - Default state of a variable is specified as none (rather than shared)
 - Signals error if not all variables are specified as shared or private

Example



```
matrixMUL.cpp  
1  #include <omp.h>  
2  #include <cstring>  
3  #include <iostream>  
4  #include <assert.h>  
5  #include "Timer.h"  
6  
7  using namespace std;  
8  
9  #define mx 1002  
10  
11  int a[mx][mx];  
12  int b[mx][mx];  
13  int c[mx][mx];  
14  int d[mx][mx];  
15  
16  void generate_matrix(int n)  
17  {  
18      int i,j;  
19      for(i=0;i<n;i++)  
20      {  
21          for(j=0;j<n;j++)  
22          {  
23              a[i][j]=rand()%100;  
24              b[i][j]=rand()%100;  
25          }  
26      }  
27      /*  
28      for(i=0;i<n;i++)  
29      {  
30          for(j=0;j<n;j++)  
31          {  
32              cout<<a[i][j]<<" ";  
33          }  
34      }  
35      */  
36  }
```

```

34         cout<<endl;
35     }
36     cout<<endl;
37
38     for(i=0;i<n;i++)
39     {
40         for(j=0;j<n;j++)
41         {
42             cout<<b[i][j]<<" ";
43         }
44         cout<<endl;
45     }
46     cout<<endl;
47     */
48 }
49
50 void check(int n)
51 {
52     int i,j;
53     for(i=0;i<n;i++)

```

```

54     {
55         for(j=0;j<n;j++)
56             assert(c[i][j]==d[i][j]);
57     }
58 }
59
60 void matrix_mult_serial(int n)
61 {
62     int i,j,k;
63     for(i=0;i<n;i++)
64     {
65         for(j=0;j<n;j++)
66         {
67             for(k=0;k<n;k++)
68             {
69                 c[i][j]+=a[i][k]*b[k][j];
70             }
71         }
72     }
73     /*
74     cout<<"\nSerial Results\n";
75     for(i=0;i<n;i++)
76     {
77         for(j=0;j<n;j++)
78         {
79             cout<<c[i][j]<<" ";
80         }
81         cout<<endl;
82     }
83     cout<<endl;
84     */
85 }
86
87 void matrix_mult_parallel(int n)
88 {
89     memset(d,0,sizeof d);
90     int i,j,k;

```

```

91     #pragma omp parallel for private(i,j,k) shared(a,b,c)
92     for(i=0;i<n;i++)
93         for( j=0;j<n;j++)
94             for(k=0;k<n;k++)
95                 d[i][j]+=a[i][k]*b[k][j];
96     check(n);
97     /*
98     cout<<"\nParallel Results\n";
99     for(i=0;i<n;i++)
100     {
101         for(j=0;j<n;j++)
102         {
103             cout<<d[i][j]<<" ";
104         }
105         cout<<endl;
106     }
107     cout<<endl;
108     */
109 }
110
111 int main()
112 {
113     int n=500;
114
115     Timer seqTime("Sequential Execution: ");
116     Timer parTime("Parallel Execution: ");
117
118     generate_matrix(n);
119
120     seqTime.Start();
121     matrix_mult_serial(n);
122     seqTime.Stop();
123
124     parTime.Start();
125     matrix_mult_parallel(n);
126     parTime.Stop();
127
128     seqTime.Print();
129     parTime.Print();
130
131     return 0;
132 }

```

Output: -

```
fasih@fasih-Latitude-7280: ~/pdclabs/lab06
fasih@fasih-Latitude-7280:~$ cd pdclabs
fasih@fasih-Latitude-7280:~/pdclabs$ cd lab06
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ subl matrixMUL.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ g++ -o matrix -fopenmp matrixMUL.cpp
fasih@fasih-Latitude-7280:~/pdclabs/lab06$ ./matrix
Sequential Execution: : 512.648 msec
Parallel Execution: : 249.223 msec
fasih@fasih-Latitude-7280:~/pdclabs/lab06$
```

Lab Task:

1. Write a code for the following in OpenMP
 - o Print the Threads Number and total threads
 2. Find the Product of two matrices [N = 1000] and print the execution time. Also, find the speedup by running it.
 3. Find the Transpose of matrices [N = 512] and print the execution time. Also, find the speedup by running it.
-